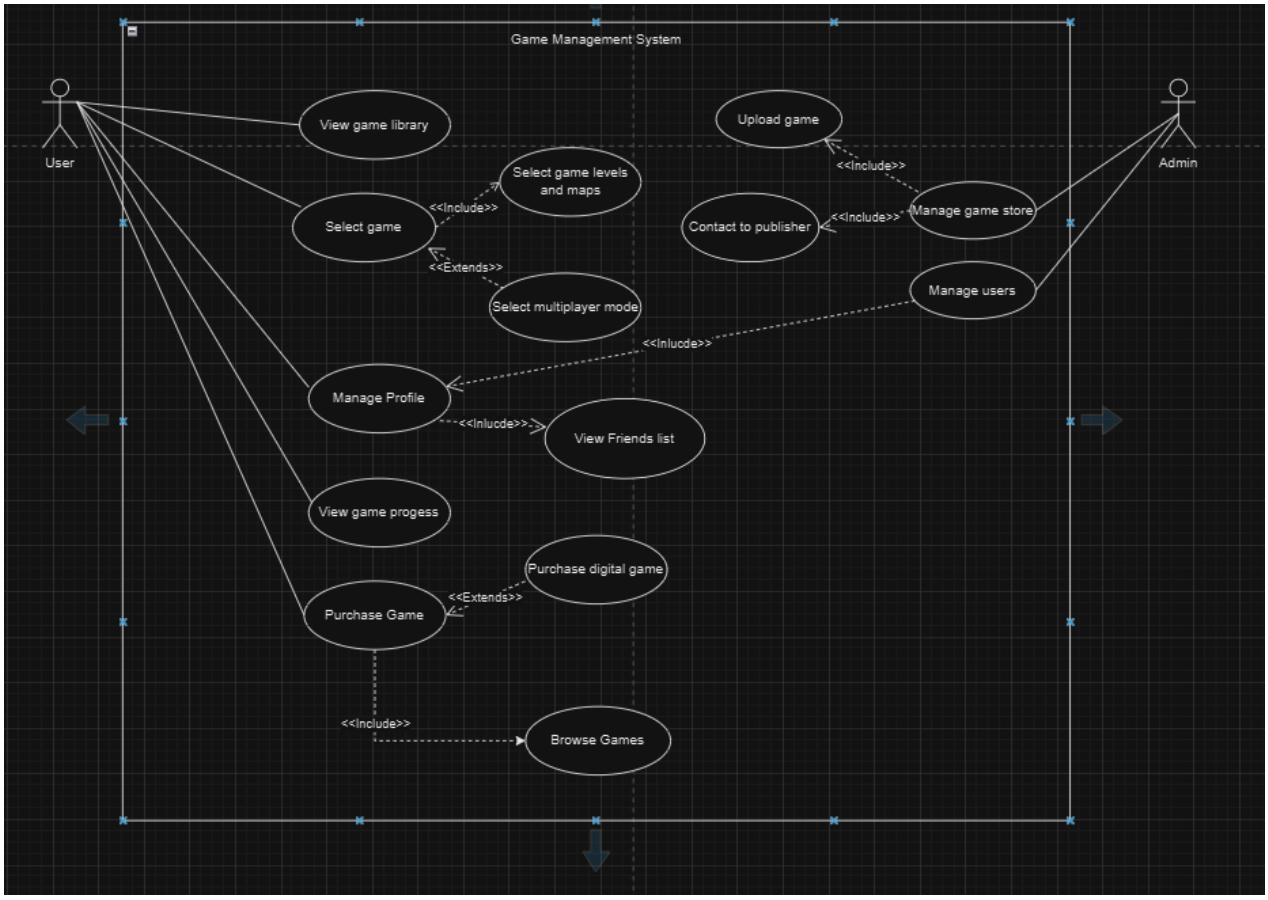
LAB 5&6 – REQUIREMENT SPECIFICATION

No.	Name	ID
1	Nguyen Dinh Khanh Ngan	ITCSIU22236
2	Phan Tran Thanh Huy	ITCSIU22056
3	Nguyen Huu Thuy	ITCSIU22141

1. Use Case Diagram



1.1. Actors

- User:
    - Represents an end-user of the game management system who can manage their game library, profile, and purchases, among other actions.
- Admin:
    - Represents a system administrator responsible for managing games, users, and the game store.

1.2. Use Cases

For User

- View Game Library:
    - Allows the user to browse the games they own.
    - Includes functionality to select specific games.
- Select Game:
    - Lets the user select a specific game from their library to play or modify.
    - Extends to functionalities such as:
        - Select Game Levels and Maps: Customize gameplay by choosing specific levels or maps.
        - Select Multiplayer Mode: Opt for multiplayer gameplay.
- Manage Profile:
    - Enables users to update their personal information.
    - Includes the ability to View Friends List to see and interact with their friends.
- View Game Progress:
    - Displays the progress made in different games by the user.
- Purchase Game:
    - Allows users to buy games from the store.
    - Extends to:
        - Purchase Digital Game: Purchase a digital version of a game.

        - Purchase Source Code: Option to purchase the source code for advanced users.
    - Includes functionality to Browse Games, enabling users to view available games before purchase.

For Admin

- Manage Game Store:
    - Provides the ability to add, modify, or remove games from the store.
    - Includes:
        - Upload Game: Add new games to the system.
- Manage Users:
    - Enables the admin to manage user accounts, such as banning users or updating roles.
- Contact Publisher:
    - Allows the admin to communicate with game publishers for support or updates.

1.3. Relationships Between Use Cases

Include Relationships:

- Select Game includes:
    - Select Game Levels and Maps
    - Select Multiplayer Mode
- Manage Profile includes:
    - View Friends List
- Purchase Game includes:
    - Browse Games

Extend Relationships:

- Purchase Game extends:
    - Purchase Digital Game
    - Purchase Source Code

2. Use Case Description

2.1. Use Case: View Game Library

Title: View Game Library

Identifier: UC1

Inputs: User selection to view the library.

Outputs: User selection to view the library.

Basic Course

Actor: User (Player)	**System**
User selects "View Game Library."	System retrieves the user's game library.
	System displays the list of games

Precondition: The user must be logged in and have at least one game in the library

Post condition: The game library is shown to the user

Alternative Course:

If the user has no games, the system displays: "Your library is empty"

User Story: As a user, I want to view my game library to see the games I own.

2.2. Use Case: Select Game

Title: Select Game

Identifier:

Inputs: Selection of a game

Outputs: Display of game-specific options (e.g., play, update)

Basic Course

Actor: User (Player)	**System**
User selects a game from the library	System fetches details of the selected game

	System displays the available actions for the game

Precondition: The user must have games in their library

Post condition: The selected game's details are displayed

Alternative Course:

If the game cannot be found, display an error message

User Story: As a user, I want to select a specific game from my library to play or manage it.

2.3. Use Case: Select Game Levels and Maps

Title: Select Game Levels and Maps

Identifier: UC3

Inputs: Selection of levels or maps for a game

Outputs: Confirmation of the selected level or map

Basic Course

Actor: User (Player)	**System**
User selects "Game Levels and Maps."	System fetches available levels/maps for the game
User selects a level/map	System saves the selection

Precondition: The selected game must support level or map selection

Post condition: Selected levels/maps are ready for gameplay

Alternative Course:

If no levels/maps are available, display "No levels or maps found."

User Story: As a user, I want to choose levels and maps to customize my gameplay experience

2.4. Use Case: Select Multiplayer Mode

Title: Select Multiplayer Mode

Identifier: UC4

Inputs: Multiplayer mode selection

Outputs: Confirmation of multiplayer mode activation

Basic Course

Actor: User (Player)	**System**
User selects "Multiplayer Mode."	System checks for available multiplayer options
User chooses a multiplayer setting or room	System initiates multiplayer mode

Precondition: The game must support multiplayer functionality

Post condition: Multiplayer mode is activated

Alternative Course:

If no multiplayer options are available, the system displays: "Multiplayer mode unavailable."

User Story: As a user, I want to enable multiplayer mode to play with others online

2.5. Use Case: Manage Profile

Title: Manage Profile

Identifier: UC5

Inputs: Updated profile information (e.g., name, email, avatar)

Outputs: Confirmation of profile updates.

Basic Course

Actor: User (Player)	**System**
User selects "Manage Profile."	System retrieves current profile information.
User updates the desired fields.	System validates and saves the changes.

Precondition: The user must be logged in.

Post condition: Profile is updated with the new information.

Alternative Course:

If validation fails (e.g., invalid email), display an error message.

User Story: As a user, I want to manage my profile to update my personal information and preferences

2.6. Use Case: View Friends List

Title: View Friends List

Identifier: UC6

Inputs: Request to view the friends list

Outputs: Display of the user's friends and their statuses

Basic Course

Actor: User (Player)	**System**
User selects "View Friends List."	System fetches the user's friends and their statuses
	System displays the list of friends

Precondition: User must have added friends to their account

Post condition: Friends list is displayed.

Alternative Course:

If the user has no friends, display: "No friends found."

User Story: As a user, I want to view my friends list to see who is online or invite them to play

2.7. Use Case: Purchase Game

Title: Purchase Game

Identifier: UC7

Inputs: Selected game and payment information

Outputs: Confirmation of the successful purchase

Basic Course

Actor: User (Player)	**System**
User selects a game to purchase	System prompts for payment details
User submits the payment information	System processes the payment and confirms the purchase

Precondition: User must be logged in and have a valid payment method

Post condition: The purchased game is added to the user's library.

Alternative Course:

If the payment fails, display an error message and allow a retry

User Story: As a user, I want to purchase a game to add it to my library

2.8. Use Case: Manage Game Store

Title: Manage Game Store

Identifier: UC8

Inputs: Game details for modification

Outputs: Confirmation of the applied changes

Basic Course

Actor: Admin	**System**
Admin selects "Manage Game Store."	System displays the list of games
Admin selects a game and performs the desired action (e.g., add, update, or remove)	System validates and applies the changes

Precondition: Admin must have appropriate permissions

Post condition: The game store is updated

Alternative Course:

If the game details are invalid, display an error message

User Story: As an admin, I want to manage the game store to add, update, or remove games

2.9. Use Case: Upload Game

Title: Upload Game

Identifier: UC9

Inputs: Game files and metadata (e.g., title, description, price)

Outputs: Confirmation of successful upload

Basic Course

Actor: Admin	**System**
Admin selects "Upload Game."	System prompts for game files and metadata
Admin uploads the required files	System validates and adds the game to the store

Precondition: Admin must be logged in and have appropriate permissions

Post condition: The game is added to the store

Alternative Course:

If validation fails, display an error message

User Story: As an admin, I want to upload new games to the store for users to purchase

2.10. Use Case: Contact Publisher

Title: Contact Publisher

Identifier: UC10

Inputs: Message content or query

Outputs: Confirmation of message sent

Basic Course

Actor: Admin	**System**
Admin selects "Contact Publisher."	System prompts for a message.
Admin enters and submits the message.	System confirms the message has been sent

Precondition: Admin must have publisher contact information

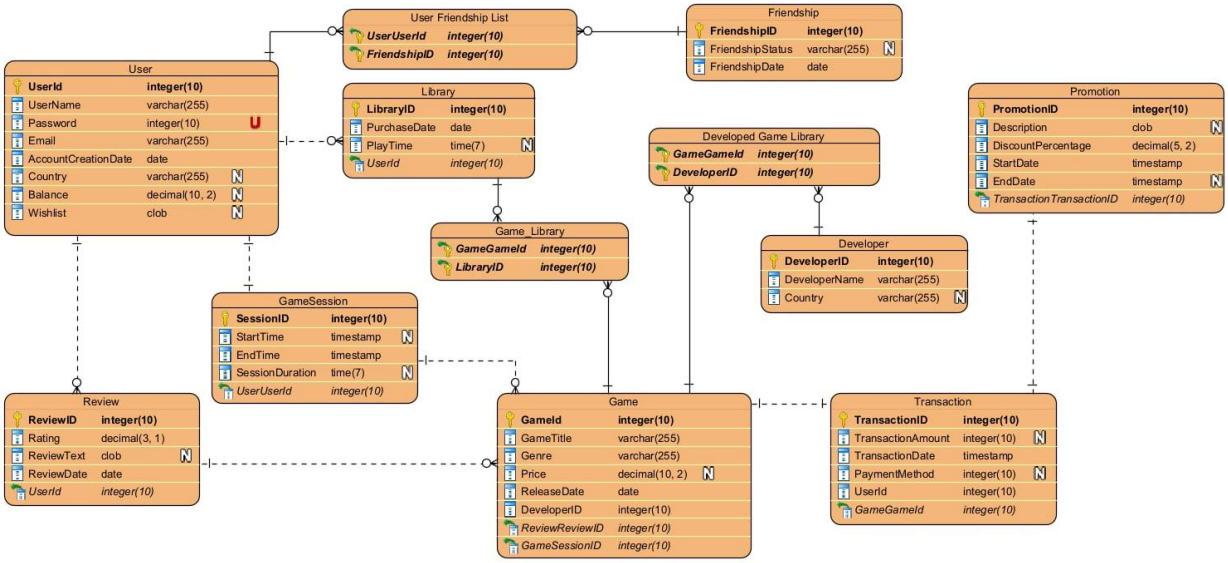
Post condition: The message is delivered to the publisher

Alternative Course:

If the message fails to send, display an error message

User Story: As an admin, I want to contact game publishers for support or updates

3. Entity Relationship Diagram



3.1. Tables and Their Roles

User

- Represents the users of the platform.
- Attributes:
    - UserId: Primary key, uniquely identifies each user.
    - Username: The user's chosen name.
    - Email: User's email address.
    - Password: Encrypted user password.
    - CreationDate: Date and time the user account was created.
    - Status: Indicates the user account's current state (e.g., active, banned).

Friendship

- Manages friend relationships between users.
- Attributes:
    - FriendshipId: Primary key for the friendship record.
    - UserOneId and UserTwoId: User IDs of the two friends.

    - FriendshipStatus: Tracks the friendship status (e.g., accepted, pending).
    - CreatedDate: The date when the friendship request was created.

Library

- Stores information about the games owned by a user.
- Attributes:
    - LibraryId: Primary key for the library record.
    - PurchaseDate: Date when the game was purchased.
    - PlayTime: Total hours spent playing the game.
- Relationships:
    - Linked to User through UserId.
    - Linked to Game through GameId.

Game

- Represents the games available on the platform.
- Attributes:
    - GameId: Primary key for the game record.
    - Title: Name of the game.
    - ReleaseDate: Date when the game was released.
    - Price: Price of the game.
    - Genre: Category of the game (e.g., RPG, action).

Developer

- Represents the game developers.
- Attributes:
    - DeveloperId: Primary key for the developer.
    - Name: Name of the developer.
    - Country: Location of the developer.
- Relationships:
    - Linked to Game through DeveloperId.

Review

- Stores user reviews for games.
- Attributes:
    - ReviewId: Primary key for the review.
    - Rating: Numerical score given by the user.
    - Comment: Text feedback about the game.
    - GameId: The ID of the game being reviewed.
    - UserId: The ID of the user who created the review.

Transaction

- Represents the purchase or financial transactions.
- Attributes:
    - TransactionId: Primary key for the transaction record.
    - Amount: The cost of the transaction.
    - Date: The date of the transaction.
    - UserId: The user who made the transaction.

GameSession

- Tracks individual game play sessions.
- Attributes:
    - SessionId: Primary key for the session.
    - StartTime and EndTime: When the session started and ended.
    - Duration: Length of the session (calculated).
    - LibraryId: Links the session to a game in the user's library.

Parameter

- Appears to store metadata or system settings for the platform.
- Attributes:
    - ParameterId: Primary key.

    - Name and Description: Key-value pairs for settings.
    - TransactionId: May link to financial parameters.

3.2. Key Relationships

User ↔ Friendship

- A user can have multiple friends, forming a many-to-many relationship via the Friendship table.

User ↔ Library ↔ Game

- A user owns multiple games via the Library.
- A game can be owned by multiple users, creating a many-to-many relationship.

Game ↔ Review ↔ User

- Users can leave reviews for multiple games.
- Each game can have reviews from multiple users.

Game ↔ Developer

- A game is developed by one developer (one-to-many relationship).

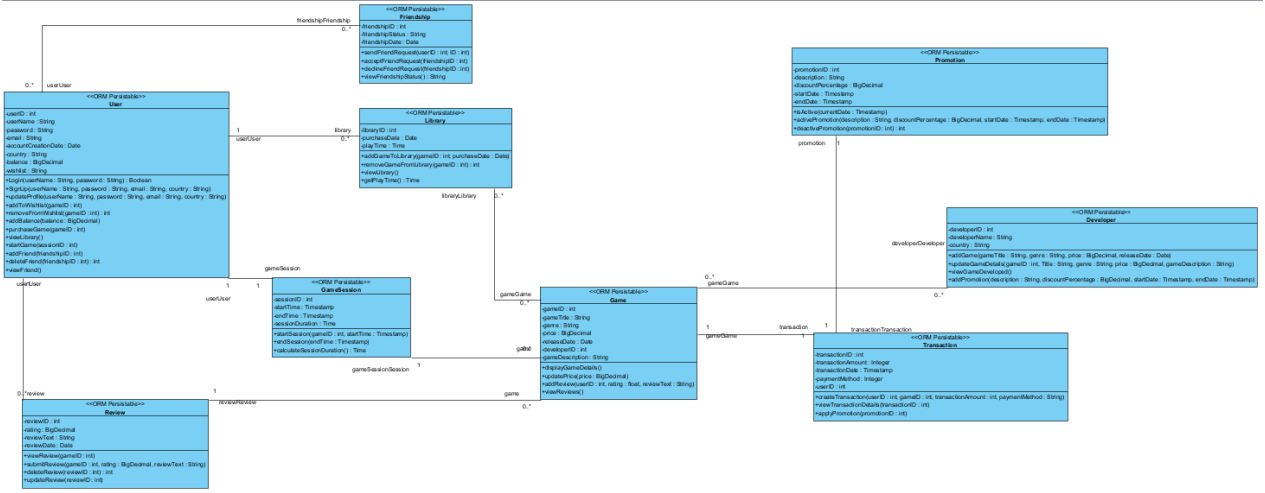
User ↔ Transaction

- A user can have multiple transactions (one-to-many relationship).

Library ↔ GameSession

- A library entry (game owned by a user) can have multiple sessions.

4. Class Diagram



4.1. Classes and Attributes:

1. User:
    - Represents the users of the platform.
    - Attributes: userID, username, email, password, accountCreationDate, country, balance, admin.
    - Methods include functionalities like signing up, logging in, adding/removing balance, and managing friendships.
2. Library:
    - Represents a user's collection of purchased games.
    - Attributes: libraryID, addTime.
    - Methods allow adding games to the library and tracking playtime.
3. Game:
    - Represents individual games available on the platform.
    - Attributes: gameID, gameTitle, genre, price, releaseDate, developerID.
    - Methods for updating price, ratings, and reviews.
4. GameSession:
    - Tracks user gaming sessions.
    - Attributes: sessionID, addTime, endTime.
    - Methods to start/end sessions and calculate session duration.

5. Review:
    - Represents reviews left by users for games.
    - Attributes: reviewID, reviewRating, reviewText, reviewTime.
    - Linked to userID and gameID.
6. Transaction:
    - Tracks transactions on the platform.
    - Attributes: transactionID, transactionAmount, paymentMethod, and gameID.
    - Methods include adding transactions and applying promotions.
7. Promotion:
    - Represents promotional offers for games.
    - Attributes: promotionID, description, discountPercentage, startDate, endDate.
    - Methods for adding and applying promotions.
8. Developer:
    - Represents game developers.
    - Attributes: developerID, developerName, country.
    - Methods for adding games and promotions.
9. Friendship:
    - Tracks friendships between users.
    - Attributes: friendshipID, friendID, requestDate, friendshipStatus.
    - Methods to handle friend requests and status.

4.2. Relationships:

- User-Library: A user can have multiple libraries, each storing their purchased games.
- Library-Game: A library is composed of multiple games.
- User-GameSession: Users can have multiple game sessions.
- User-Review-Game: A user can review multiple games, and each review is associated with a game.

- Game-Developer: Each game is developed by a specific developer.
- User-Friendship: Users can have friendships with other users.
- Transaction-Game-Promotion: Transactions are related to games, with optional promotions applied.